

Fiche de révision F# — R5.A.05 Paradigme fonctionnel

1. L'idée générale du F#

F# est un langage qui met beaucoup en avant la **programmation fonctionnelle**.

L'idée principale est de travailler avec :

- des **valeurs immuables** ;
- des **fonctions** ;
- des fonctions qui prennent d'autres fonctions en paramètre ;
- la récursivité ;
- le **pattern matching** ;
- des types comme **option** ;
- des types personnalisés avec plusieurs formes possibles.

2. **let** : déclarer une valeur

```
let x = 5
```

On associe le nom **x** à la valeur **5**.

On peut préciser le type :

```
let x: int = 5
```

Mais F# est capable de le trouver automatiquement.

```
let nom = "Gabin"  
let age = 20  
let taille = 1.80
```

F# comprend :

```
nom : string  
age : int  
taille : float
```

C'est ce qu'on appelle l'**inférence de type**.

En F#, une valeur créée avec **let** est normalement **immuable**.

3. Les types simples

```
let x: int = 5
let b: bool = true
let y: float = 3.14
let chaine: string = "toto"
let c: char = 'c'
let prix: decimal = 3.14M
```

À retenir :

- `int` : entier
- `bool` : booléen
- `float` : nombre à virgule
- `string` : chaîne de caractères
- `char` : caractère
- `decimal` : nombre décimal précis

4. Afficher quelque chose

```
printfn "Bonjour"
```

Avec interpolation :

```
let x = 5
printfn $"x vaut {x}"
printfn $"x + 1 vaut {x + 1}"
```

C'est proche de Python :

```
print(f"x vaut {x}")
```

5. Les listes

```
let maListe = [1; 2; 3; 4]
```

Type :

```
int list
```

Créer une plage :

```
let maListe = [0..10]
```

Résultat :

```
[0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10]
```

L'opérateur `::`

```
1 :: [2; 3]
```

donne :

```
[1; 2; 3]
```

`::` signifie **ajouter un élément en tête d'une liste**.

Une liste peut se voir ainsi :

```
1 :: 2 :: 3 :: []
```

6. Les tableaux

```
let tableau = [| true; false; true |]
```

Différence :

```
[1; 2; 3]      // liste  
[|1; 2; 3|]    // tableau
```

7. Les tuples

```
let personne = ("Gabin", 20)
```

Type :

```
string * int
```

Le `*` signifie ici **ET**.

Exemple :

```
let couple: int * string = (1, "roro")
```

8. Les Map

Une **Map** contient des associations clé → valeur.

```
let notes =  
    Map.ofList [  
        ("Gabin", 15)  
        ("Rémi", 12)  
    ]
```

Type :

```
Map<string, int>
```

Lire une valeur :

```
notes["Gabin"]
```

Résultat :

```
15
```

9. Les fonctions

```
let somme x y = x + y
```

Avec types :

```
let somme (x: int) (y: int) : int =  
    x + y
```

Utilisation :

```
somme 2 3
```

Résultat :

```
5
```

En F#, on écrit généralement :

```
somme 2 3
```

et non :

```
somme(2, 3)
```

10. Comprendre `int -> int -> int`

Pour :

```
let somme x y = x + y
```

F# peut afficher :

```
int -> int -> int
```

Cela peut se lire :

```
int -> (int -> int)
```

Autrement dit :

1. la fonction reçoit un `int` ;
2. elle renvoie une fonction ;
3. cette nouvelle fonction reçoit un autre `int` ;
4. elle renvoie un `int`.

C'est ce qui permet l'**application partielle**.

11. Application partielle

```
let somme x y = x + y  
let inc = somme 1
```

`inc` attend encore un paramètre.

```
inc 5
```

équivalent à :

```
somme 1 5
```

Résultat :

```
6
```

Autre exemple :

```
let produit x y = x * y  
let double = produit 2
```

```
double 5
```

donne :

10

12. Composition de fonctions >>

```
let produit x y = x * y
let double = produit 2

let somme x y = x + y
let inc = somme 1
```

Puis :

```
let x2plusun = double >> inc
```

Cela signifie :

```
x
↓
double
↓
inc
```

Donc :

```
x2plusun 5
```

donne :

11

Car :

```
double 5 = 10
inc 10 = 11
```

Équivalent :

```
let x2plusunbis x =  
    inc (double x)
```

À retenir :

```
f >> g
```

signifie :

d'abord **f**, puis **g**.

13. Les fonctions peuvent être des paramètres

```
let swap f x y =  
    f y x
```

Ici **f** est elle-même une fonction.

Exemple :

```
let puissance x n = x ** n
```

Alors :

```
swap puissance 2.0 3.0
```

fait :

```
puissance 3.0 2.0
```

Résultat :

```
9
```

C'est une **fonction d'ordre supérieur**.

Une fonction d'ordre supérieur prend une fonction en paramètre ou renvoie une fonction.

14. Les fonctions anonymes `fun`

```
fun x -> x + 1
```

Équivalent à :

```
let inc x = x + 1
```

Autre exemple :

```
let add = fun x y -> x + y
```

Comparable à :

```
x -> x + 1
```

ou en JavaScript :

```
(x) => x + 1;
```

15. Le Pattern Matching

Structure générale :

```
match valeur with  
| cas1 -> resultat1  
| cas2 -> resultat2
```

Exemple :

```
let nombre x =  
  match x with  
  | 0 -> "zéro"  
  | 1 -> "un"  
  | _ -> "autre"
```

`_` signifie :

n'importe quelle autre valeur.

16. Pattern matching sur une liste

Une liste est soit :

```
[]
```

soit :

```
head :: tail
```

Donc :

```
match liste with  
| [] -> ...  
| head :: tail -> ...
```

Avec :

```
[10; 20; 30]
```

on obtient :

```
head = 10  
tail = [20; 30]
```

17. Récursivité

Le mot-clé `rec` permet à une fonction de s'appeler elle-même.

Exemple : longueur d'une liste.

```
let rec length liste =  
  match liste with  
  | [] -> 0  
  | _ :: tail -> 1 + length tail
```

Pour :

```
length [10; 20; 30]
```

Calcul :

```
1 + length [20;30]
1 + 1 + length [30]
1 + 1 + 1 + length []
1 + 1 + 1 + 0
3
```

18. Compter les éléments non nuls

```
let rec nbNonZero liste =
  match liste with
  | [] -> 0
  | head :: tail ->
      if head = 0 then
        nbNonZero tail
      else
        1 + nbNonZero tail
```

Exemple :

```
nbNonZero [0; 5; 0; 3]
```

Résultat :

```
2
```

Comparaisons :

```
=      // égal
<>     // différent
```

19. Définir ses propres types

```
type EntreeRep = string * int
```

Cela signifie :

```
string ET int
```

Exemple :

```
let e: EntreeRep = ("Gabin", 1)
```

20. Les unions discriminées

Très important en F#.

```
type Identificateur =  
    | Name of string  
    | Id of int
```

Un **Identificateur** est donc :

```
Name contenant un string  
OU  
Id contenant un int
```

Exemples :

```
let a = Name "Gabin"  
let b = Id 42
```

Puis :

```
match a with  
| Name nom -> ...  
| Id numero -> ...
```

21. Algèbre de types

Produit : *

```
string * int
```

signifie :

```
string ET int
```

Somme : |

```
type Identificateur =  
    | Name of string  
    | Id of int
```

signifie :

```
Name string  
OU  
Id int
```

22. Le type `option`

Une valeur optionnelle est soit :

```
Some valeur
```

soit :

```
None
```

Exemple :

```
let x: int option = Some 5  
let y: int option = None
```

Cela permet d'éviter d'utiliser `null`.

23. Exemple avec `string option`

```
type Name = string * string option
```

Une personne possède :

```
nom ET surnom éventuel
```

Exemples corrects :

```
let remiS: Name =  
    ("Rémi S", Some "Panda Magique")  
  
let remiC: Name =  
    ("Rémi C", None)
```

Attention :

```
("Rémi C", "None")
```

est faux ici, car `"None"` est juste une chaîne de caractères.

24. Récupérer une valeur dans une option

```
let getPrintName x =  
    match x with  
    | _, Some surnom -> surnom  
    | nom, None -> nom
```

Exemple :

```
getPrintName ("Rémi", Some "Panda")
```

donne :

```
"Panda"
```

Et :

```
getPrintName ("Gabin", None)
```

donne :

```
"Gabin"
```

25. Pourquoi `option` change le type

```
let next x =  
    if x >= 1 then  
        None  
    else  
        Some (x + 1)
```

Cette fonction renvoie :

```
int option
```

et non :

```
int
```

Donc si :

```
let double x = x * 2
```

alors ceci ne marche pas directement :

```
double (next 0)
```

Car :

```
double attend : int
```

```
next renvoie : int option
```

Donc :

```
int ≠ int option
```

26. Extraire proprement une option

```
let getValue (x: int option) : int =  
    match x with  
    | Some value -> value  
    | None -> -1
```

Exemples :

```
getValue (Some 5)    // 5  
getValue None        // -1
```

27. Exemple Measure

```
type Measure =  
    | Sensor of float  
    | Missing
```

Une mesure est donc :

```
une valeur du capteur  
OU  
une mesure manquante
```

Exemple :

```
let serie =  
    [  
        Sensor 3.1  
        Sensor 1.43  
        Missing
```

```
Sensor 4.41  
]
```

28. Une opération générique sur des mesures

```
let op f a b =  
    match a, b with  
    | Sensor x, Sensor y -> Sensor (f x y)  
    | _ -> Missing
```

Si les deux mesures sont présentes, on applique `f`.

Sinon :

```
Missing
```

On peut ensuite créer :

```
let add =  
    op (fun x y -> x + y)  
  
let mul =  
    op (fun x y -> x * y)
```

Exemple :

```
add (Sensor 2.0) (Sensor 3.0)
```

donne :

```
Sensor 5.0
```

Mais :

```
add Missing (Sensor 3.0)
```

donne :

Missing

29. Types génériques 'a

Quand F# affiche :

```
'a
```

cela signifie :

n'importe quel type.

C'est proche de :

```
<T>
```

en Java.

Exemple :

```
let identite x = x
```

F# trouve :

```
'a -> 'a
```

Cela signifie :

prend n'importe quel type et renvoie ce même type.

30. Arbre binaire générique

```
type 'a BT =  
  | EmptyBT  
  | Node of 'a * 'a BT * 'a BT
```

Un arbre est soit :

VIDE

soit un nœud contenant :

```
valeur
ET arbre gauche
ET arbre droit
```

Exemple :

```
  5
 / \
2   8
```

peut être représenté par :

```
Node(
  5,
  Node(2, EmptyBT, EmptyBT),
  Node(8, EmptyBT, EmptyBT)
)
```

31. Rechercher dans un arbre

Version corrigée :

```
let rec isIn tree x =
  match tree with
  | EmptyBT ->
    false
  | Node (v, gauche, droite) ->
    if v = x then
      true
    else
      isIn gauche x || isIn droite x
```

Principe :

```
arbre vide
→ false
```

```

sinon :
    valeur actuelle = x ?
        oui → true
        non → chercher à gauche OU à droite

```

32. Arbre d'expression

```

type Exp =
| Var of string
| Cons of int
| Plus of Exp * Exp
| Moins of Exp * Exp
| Mult of Exp * Exp

```

Exemple :

```

x + 1

```

peut être représenté par :

```

Plus(
    Var "x",
    Cons 1
)

```

Visuellement :

```

  +
 / \
x   1

```

33. Évaluer une expression

```

let env =
    Map [
        ("x", 5)
    ]

```

Puis :

```
let rec eval e env =  
  match e with  
  | Var x ->  
    env.[x]  
  | Cons c ->  
    c  
  | Plus (left, right) ->  
    eval left env + eval right env  
  | Moins (left, right) ->  
    eval left env - eval right env  
  | Mult (left, right) ->  
    eval left env * eval right env
```

Exemple :

```
eval (Plus(Var "x", Cons 1)) env
```

donne :

```
x + 1  
x = 5  
5 + 1  
6
```

Les 7 choses à absolument comprendre

1. let

```
let x = 5
```

→ valeur.

2. Fonction

```
let somme x y =  
  x + y
```

3. Application partielle

```
let inc = somme 1
```

4. Composition

```
let h = f >> g
```

signifie :

$$h(x) = g(f(x))$$

5. Pattern matching

```
match x with
| cas1 -> ...
| cas2 -> ...
```

6. Option

```
Some valeur
None
```

7. Union discriminée

```
type Resultat =
| Succes of int
| Erreur of string
```

Tableau Java / Python / F#

Concept	Java / Python	F#
Valeur	<code>int x = 5</code>	<code>let x = 5</code>
Fonction	méthode / <code>def</code>	<code>let f x = ...</code>
Tableau	<code>[]</code> / <code>[]</code>	<code>...</code> / <code>]</code>
Liste	<code>List</code> / <code>[...]</code>	<code>[...]</code>
Lambda	<code>x -> x+1</code>	<code>fun x -> x + 1</code>
Générique	<code><T></code>	<code>'a</code>
Absence	<code>null</code> , <code>None</code>	<code>Option</code>
Switch puissant	<code>switch / match</code>	<code>match ... with</code>
Variantes de type	plusieurs classes	union discriminée

Comment lire les signatures F#

`int -> int`

Prend un `int` et renvoie un `int`.

```
let double x = x * 2
```

`int -> int -> int`

Prend deux `int` et renvoie un `int`.

```
let somme x y = x + y
```

`'a -> 'a`

Prend n'importe quel type et renvoie ce même type.

`int -> int option`

Prend un `int` et renvoie éventuellement un `int`.

`(float -> float -> float) -> Measure -> Measure -> Measure`

Prend :

1. une fonction sur deux `float` ;
2. une première `Measure` ;
3. une deuxième `Measure` ;

et renvoie une `Measure`.

Méthode pour lire du F#

Quand tu vois du F# compliqué :

1. Regarde les types

```
let f (x: int) : string =
```

`→ int -> string`

2. Regarde les paramètres

```
let f x y =
```

→ deux paramètres

3. **Cherche match** → plusieurs cas possibles

4. **Cherche rec** → récursivité

5. **Cherche Some / None** → valeur optionnelle

6. **Cherche 'a, 'b** → types génériques

Résumé ultra-court

```
let x = 5
```

→ valeur

```
let f x = x + 1
```

→ fonction

```
let f x y = x + y
```

→ fonction à deux paramètres

```
let inc = f 1
```

→ application partielle

```
f >> g
```

→ composition

```
fun x -> x + 1
```

→ fonction anonyme

```
match x with  
| ...
```

→ pattern matching

```
let rec f ...
```

→ récursivité

```
head :: tail
```

→ tête + reste d'une liste

```
Some x
```

→ valeur présente

```
None
```

→ valeur absente

```
type T =  
| A  
| B
```

→ A OU B

```
'a
```

→ type générique

```
string * int
```

→ string ET int